



Chapter 3

Command Line Parsing

3.1 **argc** and **argv**

When running a program, you may need to pass in parameters via the command line to dictate how your program will run. These parameters are known as **command line arguments**. For example, consider the following command line input:

```
./game --level 25 --difficulty 10 --multiplayer
```

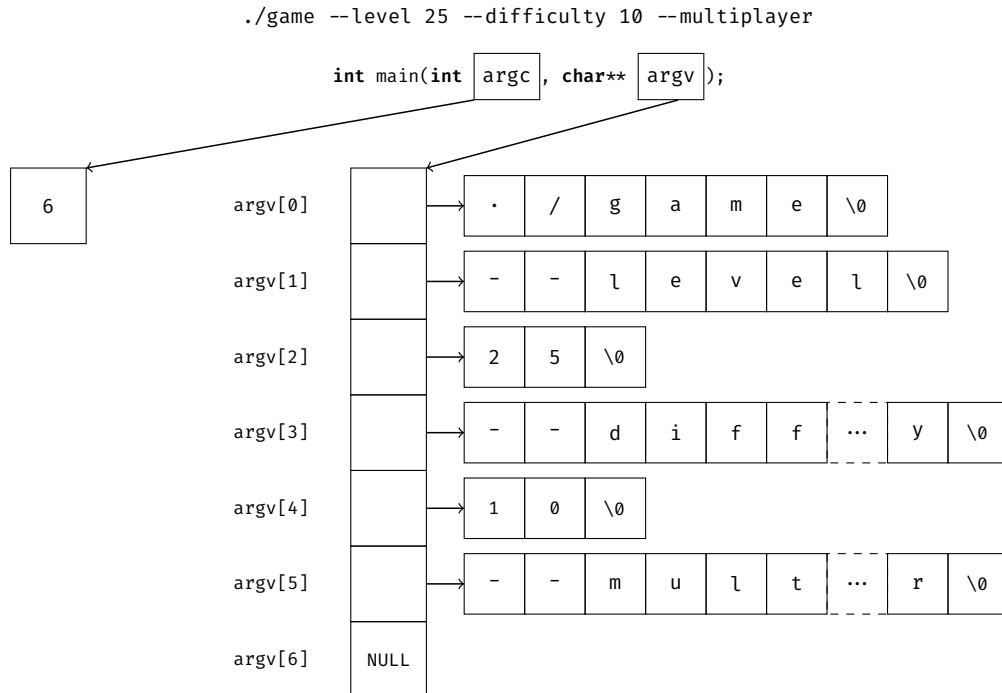
Here, the first term `./game` runs an executable named `game` in the current directory. The subsequent arguments that begin with a dash are known as **command line options**, and they can be used to specify certain features of the program. These options follow the executable name on the command line. For example, the above command may have been used to start the game at level 25, with a difficulty level of 10, and with multiplayer mode on. In this chapter, we will discuss mechanisms that allow you to handle command line options in your program.

If you want your program to support command line options, you should pass the following arguments into your `main()` function.

```
int main(int argc, char* argv[]);  
  
— or —  
  
int main(int argc, char** argv);
```

The first argument, `argc`, stores the number of arguments that are passed into the program from the command line. In the game example, there are six arguments: `./game`, `--level`, `25`, `--difficulty`, `10`, and `--multiplayer`. The second argument, `argv`, is an array of size `argc` that holds pointers to character arrays (also known as C-strings), where each C-string stores the contents of a command line argument. The name of the program itself is included as the first argument, and it is located at index 0 of the `argv` array.

A visualization of `argc` and `argv` for the example command is shown below.



All the arguments in `argv`, including the numbers, are stored as C-strings. A C-string is a character array that ends with a null character, or `'\0'` (this will be covered in more detail in chapter 16). Thus, if you want to retrieve the integer value of an argument in `argv`, you must first convert it from a C-string to an `int`. This can be done using the `atoi()` function, which accepts a C-string and returns the integer it represents (if the C-string cannot be interpreted as an integer, `atoi()` returns 0). For example, `atoi(argv[2])` returns 25 as an `int`.

Command line arguments related to input redirection are *not* included as a part of `argc` and `argv`. Consider the following example, which runs the same command as before, but with input and output files specified:

```
./game --level 25 --difficulty 10 --multiplayer < input.txt > output.txt
```

In this example, the arguments pertaining to input and output redirection are not considered as a part of `argc` and `argv`. The value of `argc` is still 6, and the `argv` array will *not* store `"<"`, `"input.txt"`, `">"`, or `"output.txt"`. Simply put, the values of `argc` and `argv` will be the same as it was before, when redirection was not specified.

3.2 Switch Statements and Enumerated Types

※ 3.2.1 Switch Statements

Before we move forward, we will go over the **switch statement** in C++, which will be used to parse command line options in the next section. Switch statements are an alternative if-else structure for primitive types. A switch statement is not only neater than a long if-else chain, but depending on the cases involved, the compiler may also be able to optimize a switch statement to improve performance. The basic structure of a switch statement is shown below:

```

1  switch (variable_to_test)
2  {
3      case potential_value_1:
4      {
5          /* code to run */
6          break;
7      }
8      case potential_value_2:
9      {
10         /* code to run */
11         break;
12     }
13     ...
14     default:
15     {
16         /* code to run */
17     }
18 }
  
```

The variable you want to check goes into the switch statement. Just like the argument of a traditional if statement, the value of this variable dictates what code gets run.

These are potential values that the variable you want to test can take on. If the value of the variable is `potential_value_1`, for example, the code under the first case statement runs.

The break statements terminates the switch statement. If not included, the program falls through and runs code in the next case.

You can have any number of cases, or values that the variable you're testing can take on.

If the value of the variable matches none of the provided cases, the code associated with the default case label runs (this is kind of like the else case of an if-else statement).

To write a switch statement, pass in the variable whose value should be checked. This variable will be tested for equality against all the values specified in the subsequent case statements. If the value of the variable matches a case, the code associated with that case runs.

You can have as many cases as you want in a switch statement, as long as

1. the variable in the switch matches the data type of the cases
2. the cases are constant expressions that can be interpreted as integer values

Each case statement starts with the `case` keyword, followed by the value that the switch variable should be compared to, followed by a colon. The order of cases in a switch statement does not matter. This is different from an if-else chain, where the order of checks may influence performance (note: for if-else chains, it would be preferable to list the most likely branches first).

When the code associated with a case runs, the case will execute until a `break` statement is reached. When a `break` statement is reached, the switch statement terminates and the program continues to the code after the switch. However, if no `break` is encountered at the end of a case, the program *falls through* to subsequent cases until a `break` is found — in other words, if the value of the switch variable matches a case, the program will execute that case, as well *all cases* that come after the match until a `break` statement is reached.

Remark: C++17 introduced the `[[fallthrough]]` attribute, which can be used to explicitly indicate that a fallthrough is intentional. This is because fallthroughs are typically not expected behavior, and they are often caused by forgetting the `break` statement at the end of a case. As a result, certain compilers will issue a warning if a fallthrough case is encountered in a switch statement. The inclusion of `[[fallthrough]]` suppresses this error, and it is also good style if fallthrough behavior is actually intended.

A switch statement may also have an optional default case at the very end. This case covers the condition where the switch variable being checked does not match with any of the provided cases. Since the default case is placed at the very end, there is no need for a `break` statement because there are no cases that follow it (and thereby no risk of unintended fallthrough).

※ 3.2.2 Enums

One disadvantage of a switch statement is that they can only be applied on variables that can be interpreted as an integer. As a result, something like this would *not* work, since it is impossible to interpret a `std::string` object as an integer:

```

1  std::string day_of_week;
2  std::cin >> day_of_week;
3  switch (day_of_week)
4  {
5      case "Monday":
6      {
7          std::cout << "It's Monday!\n";
8          break;
9      }
10     case "Tuesday":
11     {
12         std::cout << "It's Tuesday!\n";
13         break;
14     }
35     case "Sunday":
36     {
37         std::cout << "It's Sunday!\n";
38         break;
39     }
40     default:
41     {
42         std::cout << "Invalid day!\n";
43     }
44 }
```

One alternative is to use an integer to represent each string (e.g., let 1 represent Monday, 2 represent Tuesday, ..., and 7 represent Sunday):

```

1  int32_t day_of_week;
2  std::cin >> day_of_week;
3  switch (day_of_week)
4  {
5      case 1:
6      {
7          std::cout << "It's Monday!\n";
8          break;
9      }
10     case 2:
11     {
12         std::cout << "It's Tuesday!\n";
13         break;
14     }
35     case 7:
36     {
37         std::cout << "It's Sunday!\n";
38         break;
39     }
40     default:
41     {
42         std::cout << "Invalid day!\n";
43     }
44 }
```

However, this approach is complicated, messy, and prone to errors. For example, does the number 1 represent Monday or Sunday? Should Sunday be represented as 0 or 7? Not only is the code ambiguous, it is also hard to read and maintain. For complicated switch statements with many cases, other programmers may have no idea what the integers refer to and can easily mess something up.

A better approach would be to use something known as an **enumerated type**, or **enum**. An enum is a user-defined type that is restricted to a certain set of values, and it allows you to use a condition's "name" in the switch statement. For example, an enum would allow you to describe the switch cases using the actual names of the days of the week (Monday, Tuesday, etc.) instead of integers (1, 2, etc.). This is allowed because an enum associates each "name" with an integer that is used for the switch cases under the hood. To define an enum, use the following syntax:

```
enum class <NAME OF ENUM> { ... VALUES OF ENUM ... }
```

For example, the following would define an enum called `Day` that takes on the values of Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, and Sunday:

```
enum class Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
```

This allows you do things like this:

```

1  Day day_of_week = Day::Friday;
2  if (day_of_week == Day::Friday) {
3      std::cout << "It's Friday!\n";
4  } // if
```

Here, `day_of_week` is an integer behind the scenes, and not a string. However, the readability of the program is maintained since the full name of each weekday is used in the code rather than an arbitrary number.

What integer does each weekday actually represent? If we were to print out the value of `Day::Friday`, for example, the number 4 is printed out. Why is `Day::Friday` equal to 4 upon initialization? The construction of an enum follows two rules:

- If the first enum value is not explicitly assigned a value, it is assigned an integer value of 0.
- For any subsequent enum not explicitly assigned a value, its value is set to a value one greater than the value of the previous enum.

Thus, in the following enum definition:

```
enum class Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
```

Monday gets assigned the integer 0, Tuesday gets assigned the integer 1, Wednesday gets assigned the integer 2, Thursday gets assigned the integer 3, Friday gets assigned the integer 4, Saturday gets assigned the integer 5, and Sunday gets assigned the integer 6.

However, this is customizable: you can explicitly assign the value of an enum using an initializer in its definition. For instance, if you wanted Monday to be assigned the integer 1, Tuesday to be assigned the integer 2, and so on, you can explicitly assign these values as follows:

```
enum class Day { Monday = 1, Tuesday = 2, Wednesday = 3,
    Thursday = 4, Friday = 5, Saturday = 6, Sunday = 7 };
```

However, via the second rule of enum initialization, only Monday needs to be explicitly set to 1 if you want the above configuration. This is because each unassigned enum is assigned the value of the previous enum plus one (so Tuesday gets automatically assigned to $1 + 1 = 2$, Wednesday to $2 + 1 = 3$, etc.). The definition below does the same thing as the one above:

```
enum class Day { Monday = 1, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
```

It is possible for two enum values can be assigned to the same integer, and they can be compared just like integers.

You can also change the underlying integer type of an enum. For instance, the following would store the days of the week as integers of type `int8_t` (and would take up 1 byte of memory instead of 4 bytes):

```
enum class Day : int8_t { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
```

Since an object of type `char` is actually a 1-byte number behind the scenes, you can assign each enum to a character and use them in the cases of switch statements as well.

```
enum class Day : char { Monday = 'm', Tuesday = 't', Wednesday = 'w',
    Thursday = 'h', Friday = 'f', Saturday = 's', Sunday = 'u' };
```

Putting this all together, you can use enumerated types to build a cleaner, more readable switch statement, such as the one below.

```
1  enum class Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
2  Day day_of_week = ...;
3  switch (day_of_week)
4  {
5      case Day::Monday:
6      {
7          std::cout << "It's Monday!\n";
8          break;
9      }
10     case Day::Tuesday:
11     {
12         std::cout << "It's Tuesday!\n";
13         break;
14     }
15     ...
35     case Day::Sunday:
36     {
37         std::cout << "It's Sunday!\n";
38         break;
39     }
40     default:
41     {
42         std::cout << "Invalid day!\n";
43     }
44 }
```

As mentioned, switch cases allow individual cases to fall through if a `break` statement is not specified in a case. This can be useful if you want to group different cases together, such as with the following example.

```
1  enum class Suit { Diamonds, Hearts, Clubs, Spades };
2  Suit current_suit = ...; // assign value to suit
3  switch (current_suit)
4  {
5      case Suit::Diamonds:
6      {
7          [[fallthrough]];
8      }
9      case Suit::Hearts:
10     {
11         std::cout << "You got a red card!\n";
12         break;
13     }
14     case Suit::Clubs:
15     {
16         [[fallthrough]];
17     }
18     case Suit::Spades:
19     {
20         std::cout << "You got a black card!\n";
21         break;
22     }
23     default:
24     {
25         std::cout << "I don't know what your card is.\n";
26     }
27 }
```

Here, if the suit ends up being Diamonds, the code associated with the Diamonds case runs. Since there is no `break` statement in the Diamonds case, the program falls through to the next case and runs the code associated with the Hearts case as well (printing "You got a red card!"). Only then does the code encounter a `break` statement, so it exits the switch without running the code for Clubs or Spades.

Here's another example of a switch statement, which gives a prognosis of the current day's weather conditions when given one of four possible weather values as an enum.

```

1  enum class Weather { partly_cloudy, sunny, rainy, overcast };
2  Weather current_weather = ...; // assign value to current weather
3  switch (current_weather)
4  {
5      case Weather::partly_cloudy:
6      {
7          std::cout << "There will be clouds in the sky.\n";
8          [[fallthrough]];
9      }
10     case Weather::sunny:
11     {
12         std::cout << "The sun will be visible in the sky.\n";
13         break;
14     }
15     case Weather::rainy:
16     {
17         std::cout << "It will rain.\n";
18         [[fallthrough]];
19     }
20     case Weather::overcast:
21     {
22         std::cout << "The sun will likely not be visible in the sky.\n";
23         break;
24     }
25     default:
26     {
27         std::cout << "Cannot identify weather\n";
28     }
29 }

```

If `current_weather` were set to `partly_cloudy`, the switch statement would run code from the `partly_cloudy` case (line 5) to the first break encountered (line 13). Thus, the following would be printed:

```

There will be clouds in the sky.
The sun will be visible in the sky.

```

If `current_weather` were set to `sunny`, the switch statement would run code from line 10 to the first break encountered, or line 13. The following would be printed:

```

The sun will be visible in the sky.

```

If `current_weather` were set to `rainy`, the switch statement would run code from line 15 to the first break encountered, or line 23. The following would be printed:

```

It will rain.
The sun will likely not be visible in the sky.

```

If `current_weather` were set to `overcast`, the switch statement would run code from line 20 to the first break encountered, or line 23. The following would be printed:

```

The sun will likely not be visible in the sky.

```

Lastly, it should be mentioned that curly braces are not always necessary for each case. The following two versions of code do the same thing and would both work:

<pre> 1 case Day::Monday: 2 { 3 std::cout << "It's Monday!\n"; 4 break; 5 } 6 case Day::Tuesday: 7 ... </pre>	<pre> 1 case Day::Monday: 2 std::cout << "It's Monday!\n"; 3 break; 4 case Day::Tuesday: 5 ... </pre>
--	--

However, the version without the curly braces does *not* allow you to initialize variables within the code for a case! Of the two implementations below, only the one of the left would work — the one on the right would not compile.

<pre> 1 case Day::Monday: 2 { 3 int num_classes = 5; // OK 4 std::cout << "It's Monday!\n"; 5 break; 6 } 7 case Day::Tuesday: 8 ... </pre>	<pre> 1 case Day::Monday: 2 int num_classes = 5; // ERROR 3 std::cout << "It's Monday!\n"; 4 break; 5 case Day::Tuesday: 6 ... </pre>
--	---

Remark: There is a difference between an enum class and a traditional enum. Technically, both of the following approaches can be used to define a brand new enum:

```
enum class <NAME OF ENUM> { ... VALUES OF ENUM ... }
enum <NAME OF ENUM> { .. VALUES OF ENUM ... }
```

However, you should always include the class keyword when creating a new enum. This is because enum classes address several bug prone issues that traditional enums do not:

1. Traditional enums implicitly convert to integers, which may not be desired by the programmer.

The compiler is not the programmer, so it should not be making decisions on the programmer's behalf (similar to why the explicit keyword is necessary). Thus, you would want to prevent implicit conversions between an enum type and its underlying integer value, which is a protection that only enum classes provide.

```
1 enum Color { Red, Green, Blue, Yellow }; // traditional enum
2
3 int main() {
4     Color enum_value = Color::Blue;
5     if (enum_value == 2) { // implicit conversion, bad!
6         std::cout << "blue" << std::endl;
7     } // if
8 }
```

```
1 enum class Color { Red, Green, Blue, Yellow }; // enum class
2
3 int main() {
4     Color enum_value = Color::Blue;
5     if (enum_value == 2) { // compiler error, good!
6         std::cout << "blue" << std::endl;
7     } // if
8 }
```

```
1 enum class Color { Red, Green, Blue, Yellow }; // enum class
2
3 int main() {
4     Color enum_value = Color::Blue;
5     if (static_cast<int32_t>(enum_value) == 2) { // okay, because explicitly cast to integer
6         std::cout << "blue" << std::endl;
7     } // if
8 }
```

2. Traditional enums export their names to the surrounding scope, which could cause clashes with other variable names.

Unlike enum classes, the enum names used in a traditional enum cannot be used in the same scope as the enum, or in any other enum. This is not ideal behavior, as shown below:

```
1 enum Color { Red, Green, Blue, Yellow }; // traditional enum
2 enum Color2 { Red, Orange, Pink, Purple }; // not allowed, "Red" already used by 1st Color enum
3
4 int main() {
5     enum Color3 { Brown, Indigo, Cyan, Teal };
6     std::string Brown = "brown"; // not allowed, "Brown" already used by enum
7 }
```

```
1 enum class Color { Red, Green, Blue, Yellow }; // enum class
2 enum class Color2 { Red, Orange, Pink, Purple }; // okay
3
4 int main() {
5     enum class Color3 { Brown, Indigo, Cyan, Teal };
6     std::string Brown = "brown"; // okay
7 }
```

3.3 Getopt Long

※ 3.3.1 Command Line Options

In the first section of this chapter, we introduced argc and argv, which can be used to retrieve information from the command line. However, once we get these command line options, we need a way to parse them and adjust our program's behavior based on which options were entered. Let's return to our original command:

```
./game --level 25 --difficulty 10 --multiplayer
```

This command tells us that we should run level 25 of the game, with difficulty set to 10, and multiplayer mode on. A tempting (but incorrect) approach would be to retrieve the level number from argv[2], the difficulty from argv[4], and check argc to determine whether multiplayer mode is turned on. However, this would fail if the options were given in a different order!

```
./game --multiplayer --level 25 --difficulty 10
```

A better approach would be to go through the `argv` array and check for each of the options. If `--level` is specified, check the following index for the number of the level. If `--multiplayer` is specified, turn on multiplayer. This approach is more likely to produce the desired behavior, but it also has its faults. What if the user forgot to input the level number after the `--level` option? What if the `--multiplayer` option could be followed by a number that represents the number of players?

To further complicate things, command lines typically support short form options as well. A *short form option* is an abbreviated form of an option, and it is preceded with a single dash rather than two dashes (the fully typed out options we currently have, such as `--level` and `--difficulty`, are known as *long form options*). For example, the short form option of `--difficulty` may be `-d`, and the short form option of `--multiplayer` may be `-m`. In other words, the command

```
./game --level 25 --difficulty 10 --multiplayer
```

can be rewritten like this for the same outcome:

```
./game -l 25 -d 10 -m
```

In most programs, short forms and long forms can be mixed together, and short forms may be combined. As long as the difficulty follows the long form `--difficulty` or the short form `-d`, and the level follows the long form `--level` or the short form `-l`, the command should be valid. For instance, the following commands should all exhibit the same behavior:

```
./game --level 25 --difficulty 10 --multiplayer      ./game --difficulty 10 -m -l 25
./game -l 25 -d 10 -m                               ./game -md 10 --level 25
```

As you can see here, command line parsing can be get quite complicated if you are forced to do so by hand! While it might be feasible to account for all possible combinations and orderings of options when the number of possible options are low, this becomes much harder as the number of options increases. The total number of possible orderings also makes it difficult to handle errors in the command line.

※ 3.3.2 Using Getopt Long

To simplify the process of parsing command line options, we can use the GNU's `getopt_long()` function, found in the `<getopt.h>` library. A possible implementation for the game example is shown below:

```
1  #include <iostream>
2  #include <getopt.h>
3
4  // sets the difficulty of the game
5  set_game_difficulty(int32_t difficulty);
6  // sets the level of the game
7  set_game_level(int32_t level);
8  // turns multiplayer mode on
9  set_multiplayer();
10
11 int main(int argc, char** argv) {
12     int opt;
13     int opt_index = 0;
14     static struct option long_opts[] = {
15         { "difficulty", required_argument, nullptr, 'd' },
16         { "level", required_argument, nullptr, 'l' },
17         { "multiplayer", no_argument, nullptr, 'm' },
18         { nullptr, 0, nullptr, '\0' }
19     };
20
21     while ((opt = getopt_long(argc, argv, "d:l:m", long_opts, &opt_index)) != -1) {
22         switch (opt)
23         {
24             case 'd':
25             {
26                 // runs if 'd' or "difficulty" is specified on the command line
27                 std::cout << "difficulty set to " << atoi(optarg) << '\n';
28                 set_game_difficulty(atoi(optarg));
29                 break;
30             } // case 'd'
31             case 'l':
32             {
33                 // runs if 'l' or "level" is specified on the command line
34                 std::cout << "level set to " << atoi(optarg) << '\n';
35                 set_game_level(atoi(optarg));
36                 break;
37             } // case 'l'
38             case 'm':
39             {
40                 // runs if 'm' or "multiplayer" is specified on the command line
41                 std::cout << "multiplayer mode turned on\n";
42                 set_multiplayer();
43                 break;
44             } // case 'm'
45             default:
46             {
47                 std::cout << "unrecognized option" << std::endl;
48                 exit(1);
49             } // default
50         } // switch
51     } // while
52 } // main()
```

There is quite a bit of information to digest here. In the following pages, we will break this code down into segments and explain the details of `getopt_long()` one segment at a time. To start off, let's look at the `long_opts[]` array:

```
14 static struct option long_opts[] = {
15     { "level", required_argument, nullptr, 'l' },
16     { "difficulty", required_argument, nullptr, 'd' },
17     { "multiplayer", no_argument, nullptr, 'm' },
18     { nullptr, 0, nullptr, '\0' }
19 };
```

The `long_opts[]` array is an array of option structs, where each option is defined as follows:

```
1 struct option {
2     const char* name;
3     int has_arg;
4     int* flag;
5     int val;
6 };
```

Each valid option that the user can enter into the command line (e.g., `-d`, `-l`, etc.) is treated as a separate option object. As shown above, each option has four member variables. Consider the first option struct, defined on line 16.

```
{ "difficulty", required_argument, nullptr, 'd' }
```

The `name` member variable stores the long version of the option as a C-string. For example, this variable would store the word "difficulty" as a C-string for the `--difficulty` command line option (as shown above).

```
{ "difficulty", required_argument, nullptr, 'd' }
```

The `has_arg` member variable stores an integer that represents whether the option needs to be followed by an argument. There are three types of options: ones that have a required argument, ones that have no argument, and ones that have an optional argument.

- An option with a *required argument* must be followed by an additional argument if it is included in the command line. For example, `--level` would be an option with a required argument, since it must be followed by the level you want to go to (e.g., `--level 25`).
- An option with *no argument* must be run on its own if specified on the command line. For example, `--multiplayer` requires no argument, since there is no need to include anything along with it.
- An option with an *optional argument* may or may not be followed by an additional argument if specified on the command line. Unlike options with required arguments, this additional argument is *not necessary* for the command to be valid. For example, if the `--multiplayer` option defaults to 2 players, but the number of players could be customized with an additional argument (e.g., `--multiplayer 3` for 3 players), then this option would have an optional argument. For optional arguments, it is important to note that *the argument following the option is the thing that is optional, and not the actual option itself!*

The value of `has_arg` can take on three possible values, `no_argument`, `required_argument`, or `optional_argument`. All three are enums that correspond to the integers 0, 1, and 2, respectively. *Make sure you specify the correct option type for each option, as mistakes here can be difficult to track down!* Because these are enums, you can simply type out the word `required_argument` or `no_argument` in the initializer list, as shown in the provided code.

```
{ "difficulty", required_argument, nullptr, 'd' }
```

Next, we have the `flag` and `val` member variables of the option object. There are two different behaviors that can happen, depending on the value assigned to `flag`. If you want a command line option to set the value of an integer variable, you can set `flag` to the address of this integer variable and `val` to the value you want to set the integer to. Consider the following example:

```
1 static int num_players = 1;
2 int main(int argc, char** argv) {
3     ...
4     static struct option long_opts[] = {
5         { "multiplayer", no_argument, &num_players, 2 },
6         ...
7     };
8     ...
9 }
```

Here, the number of players is initially set to 1. However, because the value of `flag` for the `--multiplayer` option was set to the address of `num_players` and `val` was set to 2, the value of `num_players` will be changed to 2 if the `--multiplayer` option is seen on the command line.

In this class, you will likely encounter the second behavior, which allows you to specify a short form for each option. To accomplish this, set the value of `flag` to `nullptr` and the value of `val` to the character that should be used as the option's short form. For example, the following indicates that the option "difficulty" can be represented in short form using the letter 'd'.

```
{ "difficulty", required_argument, nullptr, 'd' }
```

The last item in the array must be an option with all of its members set to zero. This item tells `getopt_long` that there are no more option choices remaining:

```
{ nullptr, 0, nullptr, '\0' }
```

You can also initialize this last option using `{0, 0, 0, 0}`, which does the same thing.

Now, let's look at the while loop after the array definition.

```
21 while ((opt = getopt_long(argc, argv, "d:l:m", long_opts, &opt_index)) != -1) {
```

Here, the `getopt_long` function parses the command line and assigns the short form of the parameter into the `opt` variable (declared on line 12). If there are no more command line arguments to read, this function would return `-1`, and the while loop would terminate.

The third argument of the `getopt_long` function is the *short options string*, which tells the function whether it should expect an additional argument after the option. In the example, we had to tell the game program what level we wanted to go to if we specify `--level` on the command line. `--level` in this case is our option, but the 25 that follows is an additional argument that must always follow `--level`. In the short options string, you would use a colon to indicate whether a short option should be followed by an argument. If an option has a required argument, its short option must be followed by a colon in the short options string. If an option has no argument, its short option is followed by nothing. If an option has an optional argument, its short option must be followed by two colons in the short options string.

The example in the sample code, `"d:l:m"`, indicates that the

- The `d` (difficulty) option has a required argument (since `d` is followed by a colon).
- The `l` (level) option has a required argument (since `l` is followed by a colon).
- The `m` (multiplayer) option has no argument (since `m` is followed by nothing).

The order of the options in the short options string does not matter. The short option strings `"l:d:m"`, `"md:l:"`, `"ml:d:"`, and `"l:md:"` all work as alternatives. You just need to ensure that `d` and `l` are followed by a colon, since these options require an additional argument.

```
22 switch (opt)
23 {
24     case 'd':
25     {
26         // runs if 'd' or "difficulty" is specified on the command line
27         std::cout << "difficulty set to " << atoi(optarg) << '\n';
28         set_game_difficulty(atoi(optarg));
29         break;
30     } // case 'd'
31     case 'l':
32     {
33         // runs if 'l' or "level" is specified on the command line
34         std::cout << "level set to " << atoi(optarg) << '\n';
35         set_game_level(atoi(optarg));
36         break;
37     } // case 'l'
38     case 'm':
39     {
40         // runs if 'm' or "multiplayer" is specified on the command line
41         std::cout << "multiplayer mode turned on\n";
42         set_multiplayer();
43         break;
44     } // case 'm'
45     default:
46     {
47         std::cout << "unrecognized option" << std::endl;
48         exit(1);
49     } // default
50 } // switch
```

In the body of the while loop, you will have a switch statement that determines what your program does when each of the options are encountered on the command line. Each case denotes the short option of a command (as shown on lines 24, 31, and 38), and the body of each case implements the actions that are taken when the option is seen.

In the case where there is an additional argument after the option, the argument is stored in a global C-string called `optarg`. For example, if `--level 25` is seen on the command line, the 25 is stored in a variable called `optarg`, which can be used in your program (an example is shown on lines 27-28). It is important to note that `optarg` is a C-string, so you need to convert `optarg` to the correct type before you use it.

For error handling, it may be useful to add an additional default case at the end of the switch statement. The code in the default section is run when the option encountered does not match any of the options that are defined in the `long_opts[]` array. In the example, the default case is included on line 46, and exits the program if an unrecognized option is provided on the command line.

Lastly, as mentioned before, the `getopt_long` function returns `-1` if there are no more options to read from the command line. However, what happens if there are more arguments that need to be read after the final option? Consider the following command:

```
./game --multiplayer --level 25 --difficulty 10 input.txt
```

Here, we added an input file to be read in by the program. However, `input.txt` is not associated with a command line option, so `getopt_long` would return `-1` before processing this last term. We would need to process these additional arguments separately. How do we know where the input file is located? The file is located at `argv[6]` above, so we would want to start processing any additional arguments starting at index 6 of the `argv` array. Finding this index though is not trivial; this command is also valid, and the input file for this command is located at `argv[5]`.

```
./game -ml 25 -d 10 input.txt
```

Luckily, `getopt_long` keeps track of this information for us. There is a global variable called `optind` that stores the index of the next element in the `argv` array that needs to be processed. Once `getopt_long` finishes processing all of the option terms, the value of `optind` can be used to determine where the remaining non-option terms begin. In the first command, `optind` would have a value of 6 after `getopt_long` finishes running, and in the second command, `optind` would have a value of 5. In general, to check if there are non-option terms that need to be processed after running `getopt_long`, you can compare the value of `optind` with the value of `argc`. If `argc > optind` evaluates to true, then there are additional terms that you will need to parse.

3.4 Boost Program Options (*)

If you ever do C++ development in industry, there is a chance that you will not use `getopt_long` to handle command line arguments due to its functional limitations. Instead, a popular alternative is `boost::program_options` in the boost library. This section is included to provide a brief overview of `boost::program_options` as an introduction to other command line parsing libraries that are available. For a complete understanding of all the functionality supported by `boost::program_options`, you are encouraged to explore this library's documentation.

A short snippet of command line parsing code using `boost::program_options` is shown below, with the same game example covered in the previous section. You do not need to understand everything in this code for now; we will go over the details shortly.

```

1  #include <iostream>
2
3  #include <boost/program_options.hpp>
4  #include <boost/program_options/errors.hpp>
5  #include <boost/program_options/options_description.hpp>
6  #include <boost/program_options/parsers.hpp>
7  #include <boost/program_options/positional_options.hpp>
8  #include <boost/program_options/value_semantic.hpp>
9  #include <boost/program_options/variables_map.hpp>
10
11 static void greet_user(const std::string& name) {
12     std::cout << "Hello " << name << "!\n\n";
13 } // greet_user()
14
15 int main(int argc, char** argv) {
16     int32_t difficulty{};
17     std::vector<int32_t> levels_to_play;
18     std::string name;
19     bool is_multiplayer;
20
21     boost::program_options::options_description command_line_options{"Program Options"};
22     command_line_options.add_options()
23         ("difficulty,d", boost::program_options::value<int32_t>(&difficulty)->required(), "Game difficulty")
24         ("level,l", boost::program_options::value<std::vector<int32_t>>(&levels_to_play)->multitoken()->composing()
25         ->zero_tokens(), "Levels to play")
26         ("name,n", boost::program_options::value<std::string>(&name)->default_value("Guest")->notifier(greet_user),
27         "Player name")
28         ("multiplayer,m", boost::program_options::bool_switch(&is_multiplayer)->default_value(false),
29         "Starts game in multiplayer mode")
30         ("help,h", "Prints out help message");
31
32     boost::program_options::variables_map options_map;
33     try {
34         boost::program_options::parsed_options parsed_result =
35             boost::program_options::parse_command_line(argc, argv, command_line_options);
36         boost::program_options::store(parsed_result, options_map);
37         if (options_map.count("help")) {
38             std::cout << command_line_options << '\n'; // prints out options in pretty format
39             return 0;
40         } // if
41         boost::program_options::notify(options_map);
42         std::ostringstream oss;
43         for (int32_t level : levels_to_play) {
44             oss << level << " ";
45         } // for level
46         std::cout << "Difficulty: " << difficulty << '\n';
47         std::cout << "Levels: " << oss.str() << '\n';
48         std::cout << "Name: " << name << '\n';
49         std::cout << "Multiplayer: " << std::boolalpha << is_multiplayer << '\n';
50     } // try
51     catch (boost::program_options::required_option& req) {
52         std::cout << "Missing required argument: " << req.get_option_name() << '\n';
53         // error handling here
54     } // catch
55     catch (boost::program_options::error& e) {
56         std::cout << "Error while parsing command line; Message=" << e.what() << '\n';
57         // error handling here
58     } // catch
59 } // main()

```

Compile and run program:

```

g++ -std=c++1z -O3 -Wall -pedantic -pthread main.cpp -lboost_program_options -o game
./game --difficulty 10 --level 5 6 7 --name Dario --multiplayer

```

Output:

```

Hello Dario!

Difficulty: 10
Levels: 5 6 7
Name: Dario
Multiplayer: true

```

The `boost::program_options::options_description` class (created on line 21) can be used to register the command line options that your program can handle. Lines 23-30 register the command line options of difficulty, level, and multiplayer (like in the example at the end of section 3.3), as well as an option to print a help message and specify a player's name.

On line 22, the `add_options()` member of the `options_description` class can be used to define the available options for a program. If you want to specify a short option for a command line option, add a comma and the letter you want to use after the long name of the option (i.e., "difficulty,d" to allow `--difficulty` and `-d` to be used interchangeably). A short option must only consist of one letter.

Additionally, notice that some of the options listed above involve three parameters (difficulty and level), while others involve two (multiplayer and help). For the ones that have two parameters, this means that no argument can be set to each of these flags (similar to how `no_argument` behaves when working with `getopt_long()`). On the other hand, options requiring an argument can be specified using `boost::program_options::value<>` with the argument's expected type (as shown with difficulty on line 23, which expects `int32_t`).

The string we used to initialize our options description ("Program Options") is known as a *caption*, and it is printed out if you try to print out the contents of the `options_description` object on line 38. The following displays the output when the `--help` command is invoked:

Compile and run program:

```
g++ -std=c++1z -O3 -Wall -pedantic -pthread main.cpp -lboost_program_options -o game
./game --help
```

Output:

```
Program Options:
-d [ --difficulty ] arg      Game difficulty
-l [ --level ] arg          Levels to play
-n [ --name ] arg (=Guest)   Player name
-m [ --multiplayer ]         Starts game in multiplayer mode
-h [ --help ]               Prints out help message
```

The caption is optional, and you can omit it while constructing an `options_description` object. If no caption is provided, printing out the `options_description` would only print out the list of valid commands without a preceding title.

Some additional features of `boost::program_options` are summarized below:

- The `bool_switch()` call on line 28 can be used to specify a switch argument, which flips on or off some functionality if specified. In this case, if `multiplier` is provided on the command line, the value of `is_multiplayer` is switched to true.
- The `notifier()` call on line 26 can be used to invoke a separate function whenever a specific command line option is encountered. In this case, "Hello <NAME>!" is printed to the console if <NAME> is passed into the command line with the `--name` option.
- The `required()` call on line 23 specifies that the user must specify a difficulty. Failure to specify a required argument will result in an error being issued when the `notify()` function is invoked on line 41 (e.g., "Missing required argument: `--difficulty`").
- The `default_value()` call on line 26 specifies a default value if a given option is not provided on the command line. In the code, `name` has a default value of "Guest" if not specified. If you want to set a value for an option that accepts a value but is not supplied one (i.e., an optional argument in the context of `getopt_long()`), you can use the `implicit_value()` call instead (e.g., `boost::program_options::value<std::string>(&name)→implicit_value("Guest")`).
- The `multitoken()` call on line 24 can be used to specify a command line option that can take in multiple arguments (e.g., `--level 2 81`). The `composing()` call on line 24 can be used to support options that are specified more than once (e.g., `--level 2 --level 81`). The `zero_tokens()` call on line 25 can be used to indicate that it is okay for a multi-argument option to be followed with zero arguments (otherwise you would get an error like "the required argument for option '`--level`' is missing").

Multiple methods can be chained together to enable more than one behavior for a single option type. For instance, the following option description indicates that `difficulty` is set to 10 if it is not specified on the command line at all, and 15 if it is specified but without a value.

```
1 (
2     "difficulty,d",
3     boost::program_options::value<int32_t>()->default_value(10)->implicit_value(15),
4     "Game difficulty"
5 )
```

Lastly, *positional arguments* are options on the command line that are not associated with a name. Consider the following command:

```
./game input1.txt input2.txt input3.txt
```

The input files at the end of the command line are positional arguments. To handle these, you should first add an entry (with a name) into the `options_description` object, much like any other argument type (a name is still needed because this is how positional arguments are categorized into groups). The type of the value should be a vector of positional arguments that you want to read in. An example is shown below:

```
1 boost::program_options::options_description command_line_options;
2 command_line_options.add_options()
3     ("input-files", boost::program_options::value<std::vector<std::string>>(), "Input files")
```

However, this option type must also be added to an object of type `boost::program_options::position_options_description` to specify that it is a positional argument. This is shown below:

```
1 boost::program_options::position_options_description positional_options;
2 positional_options.add("input-files", -1);
```

The `-1` here specifies that an unlimited number of options on the command line should be considered as input files. You can change this value to adjust the number of positional arguments that you want to group within a given category. For instance, calling `positional_options.add("input-files", 5)` would indicate to the program that the next five positional arguments should be considered as input files, but not anything else after it. This allows you to easily group positional arguments into individual categories:

```
1 boost::program_options::position_options_description positional_options;
2 positional_options.add("player-names", 2); // two player names
3 positional_options.add("input-files", -1); // rest of positional arguments are input files
```

After specifying the positional arguments, you should include them while parsing the command line as shown. Note that we are using the more advanced `command_line_parser()` method to parse our options instead of `parse_command_line()`, which we used before. This is because `parse_command_line()` is a simplified method that does not support positional arguments.

```
1 boost::program_options::variables_map options_map;
2 boost::program_options::parsed_options parsed_result =
3     boost::program_options::command_line_parser(argc, argv)
4         .options(command_line_options)
5         .positional(positional_options)
6         .run();
7 boost::program_options::store(parsed_result, options_map);
```

A full example using positional arguments is shown below:

```
1  #include <iostream>
2
3  #include <boost/program_options.hpp>
4  #include <boost/program_options/errors.hpp>
5  #include <boost/program_options/options_description.hpp>
6  #include <boost/program_options/parsers.hpp>
7  #include <boost/program_options/positional_options.hpp>
8  #include <boost/program_options/value_semantic.hpp>
9  #include <boost/program_options/variables_map.hpp>
10
11 int main(int argc, char** argv) {
12     int32_t difficulty{};
13     int32_t level{};
14     std::vector<std::string> player_names;
15     std::vector<std::string> input_files;
16
17     boost::program_options::options_description command_line_options{"Program Options"};
18     command_line_options.add_options()
19         ("difficulty,d", boost::program_options::value<int32_t>(&difficulty), "Game difficulty")
20         ("level,l", boost::program_options::value<int32_t>(&level), "Starting level")
21         ("player-names", boost::program_options::value<std::vector<std::string>>(&player_names), "Player names")
22         ("input-files", boost::program_options::value<std::vector<std::string>>(&input_files), "Input files")
23         ("help,h", "Prints out help message");
24
25     boost::program_options::positional_options_description positional_options;
26     positional_options.add("player-names", 2); // two player names
27     positional_options.add("input-files", -1); // rest of positional arguments are input files
28
29     boost::program_options::variables_map options_map;
30     try {
31         boost::program_options::parsed_options parsed_result =
32             boost::program_options::command_line_parser(argc, argv)
33                 .options(command_line_options)
34                 .positional(positional_options)
35                 .run();
36         boost::program_options::store(parsed_result, options_map);
37         if (options_map.count("help")) {
38             std::cout << command_line_options << '\n';
39             return 0;
40         } // if
41         boost::program_options::notify(options_map);
42         std::cout << "Difficulty: " << difficulty << '\n';
43         std::cout << "Level: " << level << '\n';
44         std::cout << "Player Names:\n";
45         for (const std::string& name : player_names) {
46             std::cout << name << '\n';
47         } // for name
48         std::cout << '\n';
49         std::cout << "Input Files:\n";
50         for (const std::string& file : input_files) {
51             std::cout << file << '\n';
52         } // for file
53         std::cout << '\n';
54     } // try
55     catch (boost::program_options::error& e) {
56         std::cout << "Error while parsing command line; Message=" << e.what() << '\n';
57         // error handling here
58     } // catch
59 } // main()
```

Compile and run program:

```
g++ -std=c++1z -O3 -Wall -pedantic -pthread main.cpp -lboost_program_options -o game
./game --difficulty 10 --level 25 Dario Paolugi input1.txt input2.txt input3.txt
```

Output:

```
Difficulty: 10
Level: 25
Player Names: Dario Paolugi
Input Files: input1.txt input2.txt input3.txt
```

Practice Quiz

Disclaimer: These practice questions are not official and may not have been vetted for course material. You may use these for practice, but you should prioritize official resources from current staff for a more accurate depiction of what you need to know for assignments and exams.

1. Consider the following statement on the command line. What is the value of argc?

```
./letter -q -c -l -p -o M -b apple -e orange < dictionary.txt
```

- A) 7
- B) 10
- C) 11
- D) 12
- E) 13

2. You are writing a command line program that can be used to fetch historical weather conditions for locations in the United States. When the user runs the executable for this program (`./weather`), they are able to pass in a numerical zip code using the `--zip` flag (short form `-z`), a numerical date in YYYYMMDD format using the `--date` flag (short form `-d`), and an optional `--verbose` flag that can be used to display the results in verbose mode (short form `-v`). The verbose flag does not take in any subsequent arguments. An example is shown below (this displays the weather details for zip code 48109 on May 9, 2023 in verbose mode):

```
./weather --zip 48109 --date 20230509 --verbose
```

Which of the following option objects for `getopt_long` best depicts each of these command line options?

- A) `{"zip", no_argument, nullptr, 'z'},`
`{"date", no_argument, nullptr, 'd'},`
`{"verbose", optional_argument, nullptr, 'v'}`
- B) `{"zip", required_argument, nullptr, 'z'},`
`{"date", required_argument, nullptr, 'd'},`
`{"verbose", no_argument, nullptr, 'v'}`
- C) `{"zip", required_argument, nullptr, 'z'},`
`{"date", required_argument, nullptr, 'd'},`
`{"verbose", optional_argument, nullptr, 'v'}`
- D) `{"zip", optional_argument, nullptr, 'z'},`
`{"date", optional_argument, nullptr, 'd'},`
`{"verbose", required_argument, nullptr, 'v'}`
- E) None of the above

3. For the program described in question 2, which of the following strings is the correct short options string for use with `getopt_long()`?

- A) `zdv`
- B) `z:d:v:`
- C) `zd:v`
- D) `zdv:`
- E) `z:d:v`

4. Your friend is implementing a command line program that can be used to rank users on a class forum by their number of posts. This program accepts two options on the command line: `--rank` (short form `-r`) followed by a required integer argument, and `--help` (short form `-h`) followed by no argument. This is the `long_opts[]` array that your friend came up with:

```
1 static struct option long_opts[] = {
2     { "rank", required_argument, nullptr, 'r' },
3     { "help", no_argument, nullptr, 'h' }
4     };
```

Does this work as intended? If not, what is the issue?

- A) No, the `long_opts` array cannot be declared as `static`
- B) No, the fourth member of the rank option on line 2 should be `"r:"` instead of `'r'`
- C) No, the last item of the `long_opts` must be an option with all of its members set to zero
- D) No, for more than one of the reasons above
- E) Yes, this `long_opts` array works as intended

5. What is the purpose of the `optarg` variable in the `getopt_long()` function?
- A) `optarg` stores the value of the short options string
 - B) `optarg` keeps track of how many arguments there are to process
 - C) `optarg` stores the value of the supplied argument for options that accept arguments
 - D) `optarg` holds the integer values corresponding to the short options present in the command line
 - E) None of the above

6. Consider the following `long_opts[]` array:

```

1  static struct option long_opts[] = {
2      { "apple", no_argument, nullptr, 'a' },
3      { "banana", required_argument, nullptr, 'b' },
4      { "orange", no_argument, nullptr, 'o' },
5      { "grape", required_argument, nullptr, 'g' },
6      { "pear", no_argument, nullptr, 'p' },
7      { nullptr, 0, nullptr, '\0' }
8  };

```

Which of the following commands would **NOT** be valid? Assume the executable is named `exec`.

- A) `./exec -abg 12 --orange < fruits.txt`
- B) `./exec --apple -pb fruits.txt < fruits.txt`
- C) `./exec -o -g 13 -aob pineapple > fruits.txt`
- D) `./exec -pao --grape 281 --banana grape`
- E) More than one of the above

7. Consider the following `switch` statement:

```

1  int32_t num;
2  std::cin >> num;
3  switch (num) {
4      case 1:
5          std::string str = "yes";
6          std::cout << str << '\n';
7          break;
8      case 0:
9          std::string str = "no";
10         std::cout << str << '\n';
11         break;
12     default:
13         break;
14 }

```

Which of the following statements is **TRUE** regarding this code?

- A) The code compiles, and prints "yes" if 1 is passed into standard input, and "no" if 0 is passed into standard input
- B) The code compiles, but nothing gets printed if 1 or 0 is passed into standard input
- C) The code does not compile, since it is impossible to declare variables within a `switch` statement
- D) The code does not compile, since variables declared in one case are still visible in another case since they are part of the same scope
- E) The code does not compile, since cases must be defined in ascending order

8. Consider the following code, which is partially completed. There are two valid command line options. The 'l' or "level" option has a required argument in the form of an integer, and the value of this argument is assigned to the member variable `initial_level`. The 'h' or "hard" option turns `hard_mode` on, setting the bool to true. The 'h' option has no argument. Assuming all commands are valid, complete the code below (you may use the `atoi()` method to convert a valid C-string into an integral number):

```
class Game {
    int initial_level = 0;
    bool hard_mode = false;
public:
    void get_options(int argc, char** argv);
    /* ... other members ... */
};

void Game::get_options(int argc, char** argv) {
    int option = 0;
    int index = 0;
    // TODO: Fill out the option struct
    static struct option long_opts[] = {

};

// TODO: Complete the while loop
while ((option = getopt_long(
    switch (option) {
        case 'l':

        case 'h':

    } // switch
} // while
} // get_options()
```

» « Quiz Solutions » «

1. **The correct answer is (C).** The input redirection components of "<" and "dictionary.txt" do not contribute to argc and argv. Thus, argc is the number of other items in the command line, which comes out to 11.
2. **The correct answer is (B).** Both zip and date require additional arguments to be provided if they are specified, so these two command line options should be defined with a `required_argument`. On the other hand, verbose does not take in any subsequent arguments if it is specified, so it should be defined with `no_argument`. Notice that `optional_argument` indicates that an argument is optional *if the option is specified*, which does not apply here.
3. **The correct answer is (E).** Options that require arguments (in this case, z and d) should be followed by a colon.
4. **The correct answer is (C).** The last item of the `long_opts` array should be an option with all zeros, so that `getopt_long()` knows when there are no more option choices remaining. The options array can be defined as `static`, and the additional colon is needed for the short options string within the `getopt_long` function call, but not in the option declaration itself.
5. **The correct answer is (C).** For command line options that accept arguments, `optarg` is a C-string that stores the values of these arguments.
6. **The correct answer is (A).** If banana/b or grape/g is specified on the command line, they must be followed with an additional argument. If apple/a, orange/o, or pear/p is specified, they must be followed with no additional argument. The only command for which this is not true is (A), since b is specified without an argument after it (the 12 is the argument for g). Note that there is no restriction on including an option more than once for `getopt_long()`.
7. **The correct answer is (D).** The switch statement here does not compile because the newly created variables are not scoped properly — the lifetime of the string instantiated on line 5 extends past its specific case, which is not allowed. To fix this, curly braces can be added to each case, as shown:

```

1  int32_t num;
2  std::cin >> num;
3  switch (num) {
4      case 1:
5      {
6          std::string str = "yes";
7          std::cout << str << '\n';
8          break;
9      }
10     case 0:
11     {
12         std::string str = "no";
13         std::cout << str << '\n';
14         break;
15     }
16     default:
17     {
18         break;
19     }
20 }
```

8. The completed code is shown below. The 'l' case sets `initial_level` to the value of `optarg` after converting it to an integer, and the 'h' case sets `hard_mode` to true.

```

1  class Game {
2      int initial_level = 0;
3      bool hard_mode = false;
4  public:
5      void get_options(int argc, char** argv);
6      /* ... other members ... */
7  };
8
9  void Game::get_options(int argc, char** argv) {
10     int option = 0;
11     int index = 0;
12     static struct option long_opts[] = {
13         { "level", required_argument, nullptr, 'l' },
14         { "hard", no_argument, nullptr, 'h' },
15         { nullptr, 0, nullptr, '\0' }
16     };
17
18     while ((option = getopt_long(argc, argv, "l:h", long_opts, &index)) != -1) {
19         switch (option) {
20             case 'l':
21                 initial_level = atoi(optarg);
22                 break;
23             case 'h':
24                 hard_mode = true;
25                 break;
26         } // switch
27     } // while
28 } // get_options()
```